Records

Records

- Records sind eine Sprachfunktion für „Daten-als-Daten“:
- kompakte, shallow-immutable Klassen mit automatisch generierten
    - equals, hashCode, toString
    - Accessor-Methoden
    - kanonischen (bzw. kompakten) Konstruktor.
- Seit JDK 16 final, im LTS JDK 17 alltagstauglich verfügbar.

Records

```java
public record Person(String firstName, String lastName, int age) {}
// Nutzung
var p = new Person("Ada", "Lovelace", 36);
System.out.println(p.firstName()); // "Ada"
System.out.println(p);             // Person[firstName=Ada, lastName=Lovelace, age=36]
```

Records

- Komponenten werden zu private final Feldern und public Accessors.
- equals/hashCode vergleichen den Komponentenwertzustand; toString listet Namen und Werte.
- Records sind implizit final und erweitern java.lang.Record.
- Keine zusätzlichen nicht-statischen Instanzfelder erlaubt (neben den Komponenten).
- Dürfen Interfaces implementieren, generisch sein und verschachtelte/lokale Records definieren.

Records

```
public record Rectangle(double length, double width) {}
// Compiler erzeugt ungefähr:
public final class Rectangle extends java.lang.Record {
  private final double length, width;
  public Rectangle(double length, double width) { this.length = length; this.width = widt
  public double length() { return length; }
  public double width()  { return width; }
  @Override public boolean equals(Object o) { /* Komponentenvergleich */ }
  @Override public int hashCode() { /* aus Komponenten */ }
  @Override public String toString() { return "Rectangle[length=" + length + ", width=" +
}
```

Records: Konstruktor-Stile

Dies ist
der **kanonische Stil**.
Er ist vertrauter, aber
länger.

```
public record Email(String localPart, String domain) {
  public Email(String localPart, String domain) {
    this.localPart = Objects.requireNonNull(localPart).toLowerCase(Locale.ROOT);
    this.domain    = Objects.requireNonNull(domain).toLowerCase(Locale.ROOT);
  }
}
```

Dies ist der **kompakte Stil**.
Die Werte werden
automatisch zugewiesen,
falls dies nicht im Code
passiert.

```
public record OrderId(long value) {
  public OrderId {
    if (value <= 0) throw new IllegalArgumentException("id must be > 0");
  }
}
```

Records: Use Cases

Werteobjekte

```java
public record Money(BigDecimal amount, Currency currency) {
  public Money {
    Objects.requireNonNull(amount);
    Objects.requireNonNull(currency);
    if (amount.scale() > 2) amount = amount.setScale(2, RoundingMode.HALF_UP);
    this.amount = amount;
    this.currency = currency;
  }
}
```

```java
public record AppConfig(Map<String, String> props) {
  public AppConfig {
    this.props = Map.copyOf(props); // shallow immutability schützen
  }
}
```

Konfiguration

DTOs

```java
public record UserDto(UUID id, String name) {}
public record UserCreatedEvent(UserDto user, Instant occurredAt) {}
```

Records: Use Cases

Map-Keys

```
record Point(int x, int y) {}
Map<Point, String> labels = Map.of(new Point(1, 2), "A");
```

Implementierungen von **Sealed Interfaces**

```
sealed interface Expr permits Val, Add {}
record Val(int value) implements Expr {}
record Add(Expr left, Expr right) implements Expr {}
```

Records: Wann nicht?

- **Persistente Entities** in klassischen ORMs (JPA/Hibernate), die Mutabilität, Proxies oder no-arg-Konstruktoren erwarten.
- **Reichhaltige Domänenobjekte** mit **umfangreichem Verhalten** und **invarianten internen Zuständen**, die über reine Datenhaltung hinausgehen.
- **Tiefe Objektgraphen** mit vielen veränderlichen Komponenten (Listen, Maps, Arrays) ohne defensive Kopien.

Records: Muster

```
public record Address(String street, String city) {
  public Address withCity(String newCity) { return new Address(street, newCity); }
}
```

Wither

Factories

```
public record Email(String localPart, String domain) {
  public static Email of(String address) {
    var parts = address.split("@", 2);
    if (parts.length != 2) throw new IllegalArgumentException("invalid email");
    return new Email(parts[0], parts[1]);
  }
  // Überladener, nicht-kanonischer Konstruktor
  public Email(String address) { this(of(address).localPart(), of(address).domain()); }
}
```

Sealed Classes

Sealed Classes

- Sealed Classes/Interfaces begrenzen, welche Typen eine Klasse erweitern oder ein Interface implementieren dürfen.
- Das verbessert Modellierung, Kapselung und statische Prüfungen (z. B. bei Pattern Matching).
- Nach zwei Preview-Runden in JDK 15/16 sind Sealed Types mit JDK 17 final.

Sealed Classes

Mit dem Modifizierer ***sealed*** definierst du eine abgeschlossene Hierarchie.

Jede ***permitted subclass*** muss genau einen der Modifizierer deklarieren:

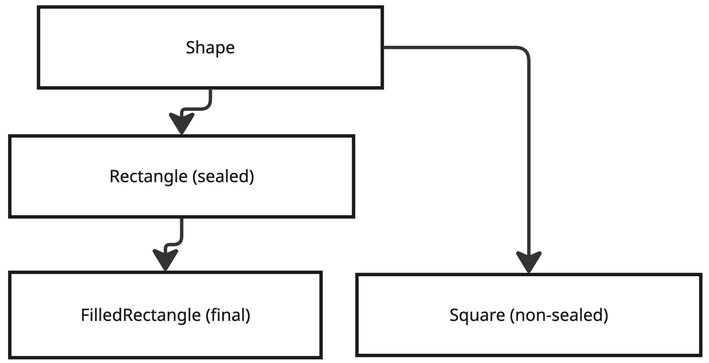
final
schließt ab

sealed
setzt fort

non-sealed
öffnet wieder

```java
public sealed interface Shape permits Circle, Rectangle, Square { }

public final class Circle implements Shape {
    private final double r;
    public Circle(double r) { this.r = r; }
    public double r() { return r; }
}

public sealed class Rectangle implements Shape permits FilledRectangle {
    private final double w, h;
    public Rectangle(double w, double h) { this.w = w; this.h = h; }
    public double w() { return w; }
    public double h() { return h; }
}

public final class FilledRectangle extends Rectangle {
    public FilledRectangle(double w, double h) { super(w, h); }
}

public non-sealed class Square implements Shape { // „bricht die Versiegelung“
    private final double a;
    public Square(double a) { this.a = a; }
    public double a() { return a; }
}
```

Über eine ***permits***-Klausel gibst du die erlaubten direkten Subklassen bzw. -interfaces an.



Außerdem gelten **Modul-/Paket-Grenzen** (gleiches Modul bzw. im Unnamed-Modul gleiches Paket).

Sealed Classes: Use Cases

```java
public sealed interface Expr permits Val, Add, Neg { }
public record Val(int value) implements Expr { }
public record Add(Expr left, Expr right) implements Expr { }
public record Neg(Expr inner) implements Expr { }
```

Algebraische Datentypen

Bibliotheken

Casts verhindern

Sealed Classes: Reflexions

```
Class<?> c = Shape.class;
boolean sealed = c.isSealed();                    // true/false
Class<?>[] subs = c.getPermittedSubclasses();     // direkte erlaubte Subtypen
```

Sealed Classes: Zusammenspiel mit Patterns

```
// JDK 17: Pattern Matching for switch
static int eval(Expr e) {
    return switch (e) {
        case Val v    -> v.value();
        case Add a    -> eval(a.left()) + eval(a.right());
        case Neg n    -> -eval(n.inner());
    };
}
```

Pattern Matching

Pattern Matching in Java 17

In JDK 17 gibt es zwei Pattern-Matching-Fähigkeiten:

1. Final: **Pattern Matching for instanceof**
2. Preview: **Pattern Matching for switch**

InstanceOf Matching

```
class MyClass{
    void myMethod() {
        Object o = getSomething();
        if (o instanceof String s && s.length() > 5) {
            System.out.println(s.substring(0, 5));
        }
        // Achtung: kein Scope rechts von ||
        if (o instanceof String s || s.length() > 5) { // kompiliert NICHT

        }
    }
}
```

Flow-Scoping-Regeln
Die Variable ist nur dort im Scope, wo der Match definitiv erfolgt ist.

Typ-Pattern sowie **Binding-Variable** für instanceof

Binding-Variable nur im wahr-Zweig möglich. Außerdem nur &&, kein ||

Pattern Matching für switch

(Preview in JDK 17, generell verfügbar in 21)

```
static String format(Object o) {
    return switch (o) {                          // switch-EXPRESSION
        case Integer i -> "int %d".formatted(i);
        case Long    l -> "long %d".formatted(l);
        case Double  d -> "double %f".formatted(d);
        case String  s -> "String %s".formatted(s);
        case null      -> "null";                // OPTIONALES null-Label
        default        -> o.toString();
    };
}
```

Baut auf **Switch Expressions** aus Java 12 auf.

case-Labels enthalten Patterns, nicht nur Konstanten. Das erweitert switch in Richtung datenorientierter Verzweigungen.

Mit Java 17 kann switch über Referenztypen arbeiten;

Guarded Pattern

```
static String triangle(Shape s) {
    return switch (s) {
        case Triangle t when (t.area() > 100) -> "Large triangle";
            case Triangle t                   -> "Small triangle";
            default                           -> "Non-triangle";
    };
}
```

Durch *when* kann man hier noch eine zusätzliche Bedingung definieren.

Dominanzprüfung

Dies kompiliert nicht, weil jeder String eine CharSequence ist.

```
class MyClass{
    void myMethod() {
        switch (obj) {
            case CharSequence cs -> System.out.println(cs.length());
            case String s        -> System.out.println(s); // FEHLER: dominiert
            default              -> {}
        }
    }
}
```

Exhaustiveness-Check für sealed

```
sealed interface S permits A, B, C {}
final class A implements S {}
final class B implements S {}
record C(int i) implements S {}

static int eval(S s) {
    return switch (s) {
        case A a -> 1;
        case B b -> 2;
        case C c -> 3;  // kein default nötig
    };
}
```

Dank sealed ist hier kein Default nötig